

Reviewer Pack — Minimal Order of Local Units in Adjoint Group and Circuit En...

Entry 9824b0275d27 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 14:44:01 UTC

Minimal Order of Local Units in Adjoint Group and Circuit Entanglement Complexity

Entry ID: 9824b0275d27 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 14:44:01 UTC

1. Conjecture

Field A (mathematical branch): Adjoint Representation Theory **Field B** (complexity object): Boolean Circuits (Circuit Complexity)

Statement:

For every n -variable Boolean function f , the minimal order of local units in the adjoint group of its characteristic polynomial φ_f is linearly correlated with its circuit entanglement complexity $e(f)$, such that $\log(\omega(\varphi_f)) = \Theta(e(f))$, where $\omega(\varphi_f)$ denotes the minimal order of local units in the adjoint group.

Rationale (proposer's reasoning):

The study of adjoint representation theory could provide insights into the algebraic structure of Boolean functions, potentially revealing a deeper connection between their algebraic properties and circuit complexity. The minimal order of local units captures an intrinsic algebraic property that might correlate with the structural entanglement in circuits computing these functions.

Taxonomy category: adjoint_representation_theory_circuit_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0201759015b618f9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an n -variable Boolean function f , if the minimal order of local units $\omega(\varphi_f)$ in its adjoint group is correlated with the circuit entanglement complexity $e(f)$, such that the Spearman's rank correlation coefficient $r \geq 0.7$ for at least 24 out of 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - adjoint representation theory AND circuit entanglement complexity - characteristic polynomial AND minimal order of local units IN BOOLEAN CIRCUITS - Boolean function ϕ_f AND $\log(\omega(\phi_f)) = \theta(e(f))$ AND adjoint group

Top relevant hits considered: - [http://arxiv.org/abs/2111.08018v2] Entanglement dynamics in hybrid quantum circuits - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1903.10609v3] Circuit Complexity of Knot States in Chern-Simons theory - [http://arxiv.org/abs/1606.00596v3] Randomized Polynomial Time Identity Testing for Noncommutative Circuits - [http://arxiv.org/abs/2305.12456v2] Recursions and characteristic polynomials of the Rows of the Circuit Array - [http://arxiv.org/abs/2009.05003v2] Strong approximation of Gaussian β -ensemble characteristic polynomials: the edge regime and the stochastic Airy functi - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1112.2313v2] QBF-Based Boolean Function Bi-Decomposition

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def characteristic_polynomial(f):
        n = len(f)
        A = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if i ^ (1 << j) < n:
                    A[i][j] = f[(i ^ (1 << j))]
                else:
                    A[i][j] = 0
        return A

    def matrix_multiplication(A, B):
        n = len(A)
```

```

C = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(n):
        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
return C

def matrix_power(A, k):
    n = len(A)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        result[i][i] = 1
    while k > 0:
        if k % 2 == 1:
            result = matrix_multiplication(result, A)
        A = matrix_multiplication(A, A)
        k //= 2
    return result

def minimal_order_of_local_units(A):
    n = len(A)
    identity = [[0] * n for _ in range(n)]
    for i in range(n):
        identity[i][i] = 1

    order = 1
    current_matrix = A
    while True:
        if matrix_power(current_matrix, order) == identity:
            return order
        order += 1

def circuit_entanglement_complexity(f):
    n = len(f)
    # Simplified heuristic for demonstration purposes
    return n * (n - 1) // 2

n = random.randint(5, 40)
f = [random.choice([0, 1]) for _ in range(1 << n)]

A = characteristic_polynomial(f)
try:
    order = minimal_order_of_local_units(A)
except Exception as e:
    return {
        "metric_name": "minimal_order_of_local_units",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": str(e)
    }

e = circuit_entanglement_complexity(f)

return {

```

```

        "metric_name": "minimal_order_of_local_units",
        "metric_value": order,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
        support_fraction = 1.0
    else:
        mean_value = None
        std_value = None
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(result["metric_value"] is not None for result in results):
        RESULT = f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}"
    else:
        first_failing_seed = next((result["seed"] for result in results if result["metric_value"] is None), None)
        RESULT = f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}"

    print(RESULT)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to provide a correlation coefficient. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14237
2	preregistration	ollama_remote	glm4:latest	0	0	9758
3	novelty	ollama_remote	glm4:latest	0	0	15609
4	novelty	ollama_remote	glm4:latest	0	0	10682
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15170
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	63571
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24420
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14287
9	judge	ollama_remote	glm4:latest	0	0	65089

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 232822 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/9824b0275d27.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/9824b0275d27.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/9824b0275d27.tar.gz* (if generated)